# TransitWatch Sofia

## Документация

**Екип: Златен Век**

React • NestJS • PostgreSQL • Docker • Kubernetes

# 1. Анализ и проучване

## 1.1 Предметна област и целева аудитория

Градският транспорт на София обслужва над 1.3 милиона души дневно чрез автобусни, трамвайни, тролейбусни и метро линии, управлявани от Столичен градски транспорт (СГТ) и Метрополитен ЕАД. Въпреки мащаба на мрежата, пътуващите разполагат с ограничени инструменти за реално-времева обратна връзка. Съществуващите канали за сигнали са насочени към администрацията, а не към останалите пътници, което означава, че критична ситуационна информация (повреди, контрольори, пренаселеност) остава затворена в бюрократични процеси вместо да достигне до хората, които имат нужда от нея в момента.

TransitWatch Sofia адресира точно този пропуск: вместо да изпраща сигнали към Центъра за градска мобилност, информацията тече директно между пътуващите в реално време. Моделът е crowdsourced - потребителите докладват, потвърждават или оспорват ситуации, като верифицираните репорти се показват на интерактивна карта.

### Целева аудитория

| Сегмент | Описание | Ключова нужда |
|---|---|---|
| Ежедневни пътници | Хора, използващи градски транспорт за работа/училище | Навременна информация преди качване |
| Студенти и ученици | Голяма група, зависима от обществен транспорт | Известия за контрольори, закъснения |
| Туристи | Посетители без познания за системата | Карта с проблеми на разбираем език |
| Активни граждани | Хора, желаещи да подобрят транспорта | Статистика, данни за системни проблеми |

*Таблица 1: Целева аудитория*

## 1.2 Преглед на съществуващи решения

За да обосновем необходимостта от TransitWatch, разглеждаме три вече съществуващи решения, които покриват части от проблемното пространство, но не решават изцяло нуждата от peer-to-peer реално-времева комуникация между пътниците.

### 1.2.1 Център за градска мобилност (ЦГМ) - Система за сигнали

ЦГМ предоставя официален канал за подаване на сигнали за проблеми в градския транспорт на София. Потребителите могат да изпращат жалби чрез уеб формуляр, телефон или приложение. Сигналите се обработват от администрацията и се препращат към съответните отговорни лица.

| Аспект | Положително | Отрицателно |
|---|---|---|
| Обхват | Официален и легитимен канал с институционална тежест | Информацията не достига до другите пътуващи |
| Скорост | Централизирана обработка | Време за реакция от часове до дни; не е real-time |
| Данни | Събира статистика за системни проблеми | Няма карта, няма интерактивност за крайния потребител |
| Достъпност | Безплатно | Липсва мобилно-оптимизиран интерфейс |

*Таблица 2: ЦГМ - Система за сигнали*

### 1.2.2 Moovit / Google Maps

Moovit и Google Maps са глобални платформи за навигация в обществен транспорт. Те предлагат маршрутизация, разписания и в определени случаи - данни за забавяния, извлечени от GPS проследяване на превозни средства.

| Аспект | Положително | Отрицателно |
|---|---|---|
| Маршрутизация | Изключително точна навигация | Не показва ситуационни проблеми |
| Данни за забавяния | GPS-базирани закъснения в реално време | Няма crowdsourced репорти от пътниците |
| UX / Интерфейс | Полиран, професионален, многоезичен | Фокус върху маршрут, не върху състояние |
| Общност | Голяма потребителска база | Няма peer-to-peer комуникация |

*Таблица 3: Moovit / Google Maps*

### 1.2.3 Facebook групи и социални мрежи

Неформални канали като Facebook групи (“Градски транспорт София”) и Twitter/X служат като ad hoc платформа за споделяне на проблеми. Потребителите публикуват снимки, коментари и жалби.

| Аспект | Положително | Отрицателно |
|---|---|---|
| Достъпност | Безплатно, голям reach | Липсва структура; информацията се губи |
| Скорост | Публикуване в реално време | - |
| Данни | Снимки и контекст от място | Няма карта, няма геолокация |
| Достоверност | Коментари и реакции като валидация | Няма системно гласуване или credibility |

*Таблица 4: Facebook групи и социални мрежи*

### Обобщение - Сравнителна матрица

| Критерий | ЦГМ | Moovit / GMaps | Facebook | TransitWatch |
|---|---|---|---|---|
| Real-time данни | Не | Частично (GPS) | Частично | Да |
| Crowdsourced репорти | Не | Не | Неструктурирано | Да |
| Интерактивна карта | Не | Да (маршрути) | Не | Да (репорти) |
| Гласуване | Не | Не | Реакции (слабо) | Да |
| Авто. изтичане | Не | N/A | Не | Да |
| Credibility | Не | Не | Не | Да |

*Таблица 5: Сравнителна матрица*

## 1.3 Аргументация за избор на технологии

Изборът на технологичен стек е продиктуван от няколко фактора: изискванията на заданието (ООП, бази данни, виртуализация), спецификата на проекта (real-time, карти, уеб), както и опита на екипа и екосистемната зрялост на инструментите.

| Слой | Технология | Аргументация |
|---|---|---|
| Frontend | React + TypeScript | Компонентна архитектура, богата екосистема, TypeScript за type safety. Leaflet за интерактивни карти. |
| Backend | NestJS + TypeScript | Модулна архитектура с вграден DI контейнер. Layered architecture по дизайн. |
| База данни | Supabase (PostgreSQL) | Релационна БД за ясни връзки. ACID транзакции. Auth, RLS, managed hosting. |
| ORM | Prisma | Type-safe заявки, автоматични миграции, интроспекция на схема. |
| Аутентикация | Supabase Auth | Email регистрация с верификация. JWT токени. RLS интеграция. |
| CI/CD | GitHub Actions | Native интеграция с GitHub repo. YAML-базирани pipelines. |
| Контейнеризация | Docker | Изолирани контейнери. Единна среда за dev и production. |
| Оркестрация | Kubernetes (k3s) | Индустриален стандарт. Container orchestration. |
| Мониторинг | Prometheus | Open-source метрики. Лесна интеграция с NestJS. |
| Secrets | GitHub + k3s Secrets | GitHub Secrets за CI/CD. k3s Secrets за production. Gitleaks pre-commit. |

*Таблица 6: Избор на технологии*

Ключово решение: PostgreSQL vs. NoSQL. Избрахме релационна база данни, защото домейн моделът има ясни релации (потребител създава репорт, репортът е свързан с линия и спирка, гласовете са свързани с репорт и потребител). Необходими са ни ACID транзакции при гласуване (за да се избегнат race conditions при едновременни потвърждения/оспорвания). PostgreSQL поддържа и геопространствени заявки чрез PostGIS разширението, което е от значение при работа с координати на спирки.

# 2. Проектиране

## 2.1 Функционални изисквания

Функционалните изисквания описват конкретните действия, които системата трябва да поддържа. Те са организирани по модули.

### FR-1: Автентикация и потребителски профил

| ID | Изискване | Приоритет |
|---|---|---|
| FR-1.1 | Потребителят може да се регистрира с email и парола чрез Supabase Auth | Висок |
| FR-1.2 | Потребителят може да влезе в системата с email и парола | Висок |
| FR-1.3 | Системата издава JWT токен при успешна автентикация | Висок |
| FR-1.4 | Всеки потребител има credibility score, който се съхранява в базата | Нисък |

*Таблица 7: Функционални изисквания: Автентикация*

### FR-2: Управление на репорти

| ID | Изискване | Приоритет |
|---|---|---|
| FR-2.1 | Автентикиран потребител може да създаде репорт за конкретна линия (POST /reports) | Висок |
| FR-2.2 | Репортът съдържа: линия (lineId), категория, описание (до 140 символа) | Висок |
| FR-2.3 | Системата изчислява expiresAt на база категорията (Strategy pattern) | Висок |
| FR-2.4 | Всеки потребител може да вижда всички активни репорти (GET /reports/active) | Висок |
| FR-2.5 | Потребителят може да филтрира репорти по линия (GET /reports/line/:lineId) | Среден |
| FR-2.6 | Потребителят може да вижда детайли за конкретен репорт (GET /reports/:id) | Среден |

| ID | Изискване | Приоритет |
|---|---|---|
| FR-2.7 | Собственикът на репорт може да го изтрие (DELETE /reports/:id) | Среден |

*Таблица 8: Функционални изисквания: Репорти*

### FR-3: Гласуване и верификация

| ID | Изискване | Приоритет |
|---|---|---|
| FR-3.1 | Автентикиран потребител може да потвърди (confirm) или оспори (dispute) репорт | Среден |
| FR-3.2 | Всеки потребител може да гласува само веднъж за даден репорт (unique constraint) | Среден |
| FR-3.3 | Credibility score на репорта се актуализира при ново гласуване | Нисък |

*Таблица 9: Функционални изисквания: Гласуване*

### FR-4: Автоматично изтичане на репорти

| ID | Изискване | Приоритет |
|---|---|---|
| FR-4.1 | Cron job проверява на всеки 5 минути за изтекли репорти | Висок |
| FR-4.2 | Изтеклите репорти автоматично се маркират със статус expired | Висок |
| FR-4.3 | Времето за изтичане е различно за всяка категория | Висок |

*Таблица 10: Функционални изисквания: Изтичане*

### FR-5: Интерактивна карта

| ID | Изискване | Приоритет |
|---|---|---|
| FR-5.1 | Началната страница показва карта на София с маркери за активни репорти | Висок |
| FR-5.2 | При натискане на маркер се показват детайли за репорта | Среден |
| FR-5.3 | Картата използва Leaflet с OpenStreetMap tiles | Висок |
| FR-5.4 | Картата показва всички линии и спирки като ежедневен тракер | Висок |

*Таблица 11: Функционални изисквания: Карта*

### FR-6: Линии и спирки

| ID | Изискване | Приоритет |
|---|---|---|
| FR-6.1 | Системата съхранява линии (bus, tram, metro, trolley) с техните спирки | Висок |
| FR-6.2 | Всяка спирка има име, координати (lat, lng) и принадлежност към линия | Висок |

*Таблица 12: Функционални изисквания: Линии и спирки*

## 2.2 Архитектура на системата

TransitWatch Sofia използва Layered Architecture (многослойна архитектура), при която всеки слой има ясно дефинирана отговорност и комуникира само със съседния слой. Това осигурява separation of concerns, testability и възможност за независима промяна на всеки слой.

| Слой | Технология | Отговорност |
|---|---|---|
| Presentation Layer | React + TypeScript, Leaflet | Потребителски интерфейс, интерактивна карта, форми за репорти |
| API Layer | NestJS Controllers | HTTP endpoint-и, валидация, маршрутизация на заявки |
| Service Layer | NestJS Services | Бизнес логика: валидация, credibility, изтичане |
| Repository Layer | Prisma + IRepository | Абстракция над БД. Repository Pattern. |
| Data Layer | Supabase (PostgreSQL) | Съхранение на данни, миграции, RLS |

*Таблица 13: Архитектура на системата*

Потокът на данните е следният: потребителят взаимодейства с React приложението, което изпраща REST API заявки към NestJS backend-а. Controller-ът делегира на Service класа, който съдържа бизнес логиката. Service класът работи с Repository интерфейс (DIP принцип), а конкретната имплементация използва Prisma за достъп до PostgreSQL базата.

### NestJS Модулна структура

| Модул | Компоненти | Описание |
|---|---|---|
| AuthModule | AuthController, AuthService, JwtStrategy | Регистрация, вход, JWT валидация |
| ReportModule | ReportController, ReportService, IReportRepository | CRUD за репорти, гласуване, изтичане |
| LineModule | LineController, LineService, LineRepository | Линии, спирки, свързани репорти |
| UserModule | UserController, UserService, UserRepository | Профили, credibility score, следвани линии |

*Таблица 14: NestJS модулна структура*

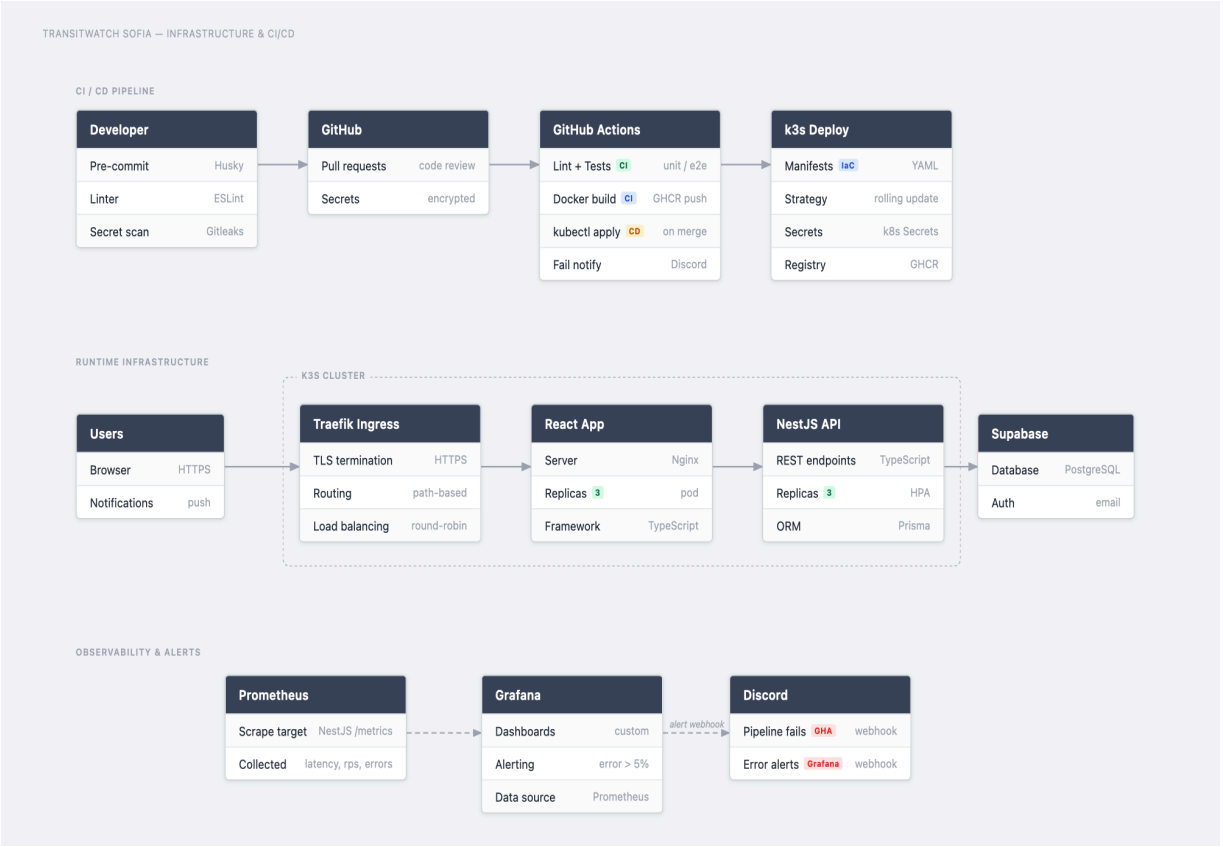
### Design Patterns

| Pattern | Къде | Какъв проблем решава |
|---|---|---|
| Repository Pattern | IReportRepository интерфейс | Абстрахира достъпа до БД. Позволява смяна на ORM без промяна на бизнес логиката. |
| Strategy Pattern | Типове репорти (enum + strategy) | Добавяне на нови категории без промяна на съществуващ код (OCP). |

*Таблица 15: Design Patterns*

## 2.3 Инфраструктурна диаграма

Инфраструктурната диаграма илюстрира пълния път на заявката от потребителя до базата данни, включително CI/CD pipeline-а и pre-commit hooks за качество на кода. Потребителят достига системата чрез Traefik Ingress, който маршрутизира трафика към React frontend или NestJS API. API слоят комуникира със Supabase PostgreSQL за съхранение на данни и с Prometheus за метрики. Паралелно, всеки push към GitHub минава през CI/CD pipeline (lint → тестове → Docker build → deploy), а pre-commit hooks (Husky + Gitleaks) гарантират качество на кода и липса на тайни преди всеки commit.




*Фигура 1: Инфраструктурна диаграма*

## Описание на компонентите

| Компонент | Описание | Конфигурация |
|---|---|---|
| Traefik Ingress | Reverse proxy и load balancer | Вградена в k3s |
| Web Deployment | React SPA, сервирана от Nginx | 3 replicas, port 80 |
| API Deployment | NestJS REST API сървър | 3 replicas, HPA (3-10), port 3000 |
| Prometheus | Събира метрики от API pods | Alerting при error rate > 5% |
| GitHub Actions | CI/CD pipeline: lint → test → build → deploy | Trigger: push to main, PR |
| Discord Webhook | Нотификации при failure или alerts | - |

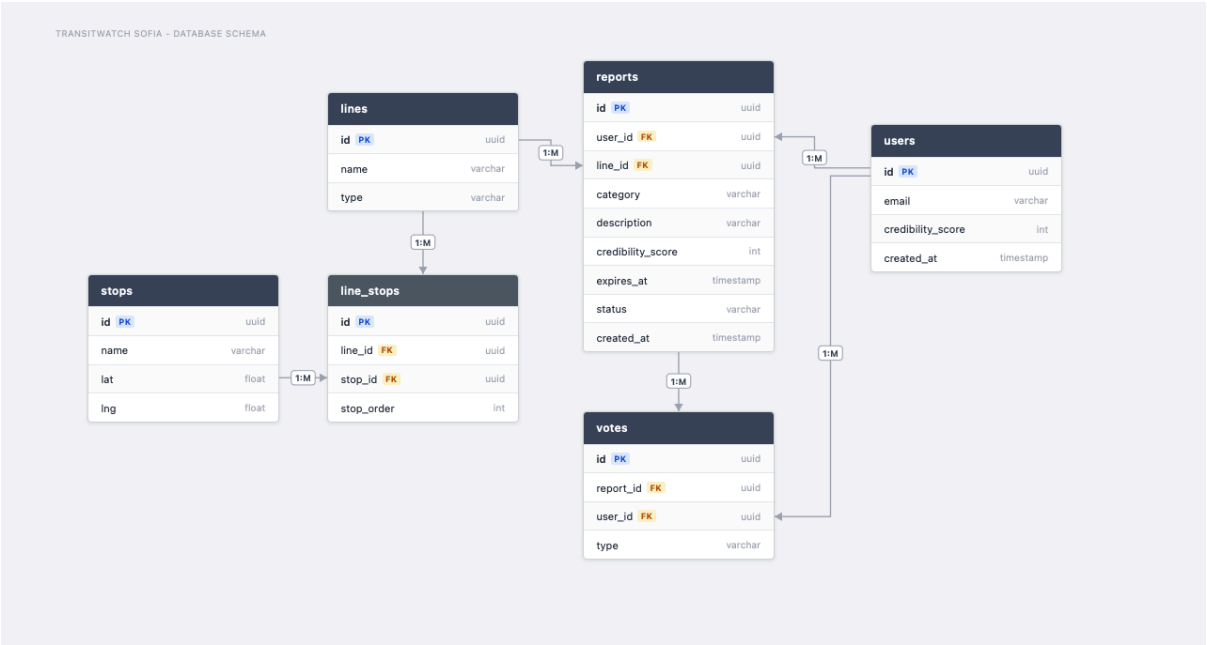
*Таблица 16: Компоненти на инфраструктурата*

## 2.4 Схема на базата данни

Базата данни използва PostgreSQL чрез Supabase и е моделирана с Prisma ORM. Схемата съдържа 6 таблици с ясно дефинирани връзки. Връзката между линии и спирки е реализирана чрез свързваща таблица line_stops (много-към-много), което позволява една спирка да принадлежи на много линии.

## ER диаграма (Entity-Relationship)

Базата данни е релационна (PostgreSQL чрез Supabase) и е нормализирана до 3NF. Всяка таблица съдържа само атрибути, зависещи директно от първичния ключ - няма транзитивни зависимости и няма повторение на данни.




*Фигура 2: Схема на базата данни (ER диаграма)*

## Описание на таблиците

### Таблица users

Регистрирани потребители на системата

| Колона | Тип | Ограничения | Описание |
|---|---|---|---|
| id | UUID | PK | Уникален идентификатор (от Supabase Auth) |
| email | TEXT | UNIQUE, NOT NULL | Email адрес на потребителя |
| credibility_score | INT | DEFAULT 0 | Рейтинг на достоверност |
| created_at | TIMESTAMP | DEFAULT now() | Дата на регистрация |

*Таблица 17: Таблица users*

### Таблица lines

Линии на градския транспорт

| Колона | Тип | Ограничения | Описание |
|---|---|---|---|
| id | UUID | PK | Уникален идентификатор |
| name | VARCHAR | NOT NULL | Име/номер на линията (напр. “23”, “5”) |
| type | VARCHAR | NOT NULL | Вид транспорт: bus, tram, metro, trolley |

*Таблица 18: Таблица lines*

### Таблица stops

Спирки на градския транспорт

| Колона | Тип | Ограничения | Описание |
|---|---|---|---|
| id | UUID | PK | Уникален идентификатор |
| name | VARCHAR | NOT NULL | Име на спирката |
| lat | FLOAT | NOT NULL | Географска ширина |
| lng | FLOAT | NOT NULL | Географска дължина |

*Таблица 19: Таблица stops*

### Таблица line_stops

Спирките на всяка линия

| Колона | Тип | Ограничения | Описание |
|---|---|---|---|
| id | UUID | PK | Уникален идентификатор |
| line_id | UUID | FK → lines | Линия |
| stop_id | UUID | FK → stops | Спирка |
| stop_order | INT | NOT NULL | Пореден номер на спирката по линията |

*Таблица 20: Таблица line_stops*

### Таблица reports

Потребителски репорти за проблеми

| Колона | Тип | Ограничения | Описание |
|---|---|---|---|
| id | UUID | PK | Уникален идентификатор |
| user_id | UUID | FK → users | Автор на репорта |
| line_id | UUID | FK → lines | Линия, за която е репортът |
| category | VARCHAR | NOT NULL | Категория (VEHICLE_ISSUE, TRAFFIC, INSPECTORS, SAFETY, OTHER) |
| description | VARCHAR | max 140 | Кратко описание на проблема |
| credibility_score | INT | DEFAULT 5 | Текущ рейтинг на репорта |
| expires_at | TIMESTAMP | NOT NULL | Кога изтича автоматично (Strategy) |
| status | TIMESTAMP | NOT NULL | Статус на репорта |
| created_at | TIMESTAMP | DEFAULT now() | Дата на създаване |

*Таблица 21: Таблица reports*

### Таблица votes

Гласове за потвърждение/оспорване на репорти

| Колона | Тип | Ограничения | Описание |
|---|---|---|---|
| id | UUID | PK | Уникален идентификатор |
| report_id | UUID | FK → reports | Репорт, за който е гласът |
| user_id | UUID | FK → users | Потребител, който гласува |

| Колона | Тип | Ограничения | Описание |
|---|---|---|---|
| type | VARCHAR | NOT NULL | Вид глас: confirm или dispute |
| - | - | UNIQUE(report_id, user_id) | Един потребител - един глас на репорт |

*Таблица 22: Таблица votes*

### Връзки между таблиците

| Връзка | Тип | Описание |
|---|---|---|
| users → reports | 1:N | Потребител създава много репорти |
| users → votes | 1:N | Потребител подава много гласове |
| lines → line_stops | 1:N | Линия има много записи в line_stops |
| stops → line_stops | 1:N | Спирка присъства в много линии чрез line_stops |
| lines → stops | M:M | Много-към-много чрез line_stops |
| lines → reports | 1:N | Линия може да има много репорти |
| reports → votes | 1:N | Репорт може да има много гласове |

*Таблица 23: Връзки между таблиците*

## 2.5 UML Class диаграма

UML класовата диаграма показва основните класове на системата, техните атрибути, методи и връзки. Диаграмата демонстрира наследяване (BaseEntity е базов клас за User, Report, Stop, Line, Vote), композиция (Report съдържа Line), асоциация (User създава Report), и Dependency Inversion (ReportService зависи от IReportRepository интерфейс). Strategy Pattern е представен чрез IReportStrategy интерфейс и отделни стратегии за всяка категория репорт.

*Фигура 3: UML класова диаграма*